

Lab 2 Technical Report

Merging Login/Register and Employee CRUD into a Unified Enterprise Application

Student ID: 24-57480-2

Solution / App Name: 24-57480-2_CompanyApp

Target Framework: .NET Framework 4.8

Database: Microsoft SQL Server LocalDB (dbCompanyApp)

GitHub Repository: <https://github.com/yeanfokir/Employee-Management-System.git>

Submission Date: August 2026

1. Executive Summary & Architectural Transformation

This engineering report details the complete refactoring, migration, and architectural consolidation of two distinct legacy applications into a production-grade, enterprise Windows Forms solution.

Initial State: The starting codebase consisted of two disconnected solutions:

- **Login-and-Register Solution:** Targeted .NET Framework 4.7.2, used `System.Data.OleDb` connected to a Microsoft Access database (`db_users.mdb`) stored in `bin\Debug`, and had architectural vulnerabilities including SQL injection, class-level connection leaks, and unclosed background processes.
- **EmployeeDetails Solution:** Targeted .NET Framework 4.8, used `System.Data.SqlClient` connected to SQL Server LocalDB, containing basic CRUD operations in a single form without authentication or user tracking.

Consolidated Final State: The resulting project, `CompanyApp` (Solution: `24-57480-2_CompanyApp.sln`), establishes a unified .NET Framework 4.8 architecture powered by a single centralized SQL Server database (`dbCompanyApp`). Application security guarantees that users must authenticate before reaching protected CRUD features, session state tracks active credentials, and every created employee record is permanently linked to its creator via relational integrity.

Key Architectural Achievement: 100% removal of legacy OleDb / MS Access dependencies, zero orphaned processes upon logout/exit, and unified relational queries displaying creator ownership via SQL `LEFT JOIN`.

2. The Six Structural Conflicts & Engineering Solutions

#	Conflict Area	Root Cause & Hazard	Engineering Resolution
1	Namespace Divergence	<code>Login_and_Register</code> vs <code>EmployeeDetails</code> . Pasting forms directly produced compilation errors (CS0260) because partial classes were defined in mismatching namespaces.	Standardized root namespace to <code>EmployeeDetails</code> across all <code>.cs</code> and <code>.Designer.cs</code> files.
2	Data Provider Incompatibility	<code>OleDbConnection</code> cannot connect to SQL Server, and <code>SqlConnection</code> cannot open Access <code>.mdb</code> files.	Completely removed <code>System.Data.OleDb</code> and rewrote data access using <code>System.Data.SqlClient</code> with parameterized queries (<code>@param</code>).
3	Dual Database Segregation	Two separate database engines (Access vs LocalDB) prevented relational joins and data integrity.	Designed a unified SQL Server database (<code>dbCompanyApp</code>) hosting both <code>Users</code> and <code>Emp_details</code> tables linked via Foreign Key.
4	Framework Version Divergence	Target version mismatch (v4.7.2 vs v4.8).	Configured host project to target <code>.NET Framework 4.8</code> , ensuring binary safety when importing older components.
5	Dual Entry Points (CS0017)	Two <code>Program.cs</code> files with two <code>Main()</code> methods violated C# single entry-point rules.	Retained a single <code>Program.cs</code> calling <code>Application.Run(new frmLogin())</code> , enforcing login-first security.
6	Hidden File Dependency	<code>db_users.mdb</code> existed only in <code>bin\Debug</code> ; running "Clean Solution" broke login capabilities.	Extracted all user accounts and migrated them to <code>dbo.Users</code> in SQL Server, providing deterministic deployment via <code>Schema.sql</code> .

3. Unified Database Design & Data Migration

The unified database `dbCompanyApp` eliminates data fragmentation. Relational integrity is established between employee records and authenticating users.

Relational Schema Definition:

```
-- 1. Users Table
CREATE TABLE dbo.Users (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    Username NVARCHAR(50) NOT NULL UNIQUE,
    Password NVARCHAR(200) NOT NULL,
    Role NVARCHAR(50) DEFAULT 'User' NOT NULL,
    CreatedAt DATETIME DEFAULT GETDATE() NOT NULL
);

-- 2. Employee Details Table (Relational Foreign Key)
CREATE TABLE dbo.Emp_details (
    EmpId NVARCHAR(50) PRIMARY KEY,
    EmpName NVARCHAR(100) NOT NULL,
    EmpAge INT NOT NULL,
    EmpContact NVARCHAR(20) NULL,
    EmpGender NVARCHAR(10) NULL,
    CreatedBy INT NULL,
    CONSTRAINT FK_Emp_CreatedBy FOREIGN KEY (CreatedBy) REFERENCES dbo.Users(UserID)
);

-- 3. Audit Activity Log (Bonus Feature)
CREATE TABLE dbo.LoginHistory (
    LogId INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT NULL,
    Username NVARCHAR(50) NOT NULL,
    Action NVARCHAR(50) NOT NULL,
    Timestamp DATETIME DEFAULT GETDATE() NOT NULL,
    CONSTRAINT FK_LoginHistory_User FOREIGN KEY (UserID) REFERENCES dbo.Users(UserID)
);
```

Access Data Migration:

All accounts from the legacy MS Access database (`db_users.mdb`) were extracted and seeded into `dbo.Users`:

- `admin` / `12345` (Role: Admin)
- `sayan` / `Sayan@1234` (Role: User)

- `sayanchamp` / `12345` (Role: User)
- `sachin` / `123` (Role: User)
- `sayan_admin` / `12345` (Role: Admin)
- `ifte` / `123` (Role: User)
- `ifte` / `123` (Role: User)

Design Rationale on Foreign Key Nullability: The `CreatedBy` column in `dbo.Emp_details` is intentionally defined as `NULL`-able. This allows legacy employee records (imported without a known author) to coexist seamlessly with newly created employee records without violating relational integrity constraints.

4. The Three-File Rule & Form Import Methodology

In Windows Forms, every form is not a single file, but a triad of tightly coupled files:

1. `<Form>.cs` : Contains event handlers, business logic, and class methods.
2. `<Form>.Designer.cs` : Contains designer-generated control instantiations, property definitions, and component tree hierarchies.
3. `<Form>.resx` : Contains XML/binary resources, layout metadata, and icon definitions.

Import Strategy: All 9 files for `frmLogin`, `frmRegister`, and `frmDashboard` were imported into the host project. In `CompanyApp.csproj`, files were nested using `<DependentUpon>` elements so that Visual Studio groups them under their respective parent form. All namespaces across both `.cs` and `.Designer.cs` were refactored to `EmployeeDetails` to resolve partial class declarations.

5. Porting OleDb to SqlClient (User.cs & Session.cs)

The legacy login implementation suffered from severe architectural flaws: it concatenated user input directly into SQL strings (vulnerable to SQL injection), held unmanaged class-level connections open (causing runtime crashes on second login), and used incorrect boolean logic (`&&` instead of `||`) during registration validation.

Engineered Solution:

- **Single Connection Configuration:** All SQL connections reference `App.config` via `ConfigurationManager.ConnectionStrings["connString"].ConnectionString`.

- **Consistent Data Access (`User.cs`):** Designed to match `Employee.cs`, implementing parameterized methods: `ValidateLogin(username, password)`, `UsernameExists(username)`, `RegisterUser(...)`, and `LogActivity(...)`.
- **Session Container (`Session.cs`):** Static container storing `UserID`, `Username`, and `Role` in memory during the application lifecycle.

6. Application Lifecycle & Workflow Wiring

The application workflow ensures complete access control and memory management:

1. **Entry Point:** `Program.cs` executes `Application.Run(new frmLogin())`.
2. **Authentication:** `frmLogin` verifies credentials. On success, sets `Session.UserID` and `Session.Username`, displays `frmDashboard`, and hides itself.
3. **Employee Management:** The Dashboard offers a "MANAGE EMPLOYEES" button that opens `frmEmployee` as a modal dialog.
4. **Ownership Stamping:** When an employee is added, `employee.CreatedBy = Session.UserID` is stamped into the database.
5. **Relational Visualization (LEFT JOIN):** The employee grid executes:

```
SELECT e.EmpId, e.EmpName, e.EmpAge, e.EmpContact, e.EmpGender,  
       ISNULL(u.Username, 'Migrated / N/A') AS [CreatedBy]  
FROM dbo.Emp_details e  
LEFT JOIN dbo.Users u ON e.CreatedBy = u.UserID;
```

6. **Safe Logout:** Clicking "LOGOUT" prompts a confirmation dialog, clears session data via `Session.Clear()`, shows a fresh `frmLogin`, and closes the dashboard. Closing the login form cleanly executes `Application.Exit()` to eliminate orphaned background processes.

7. Why One Database is Superior to Two (Architectural Rationale)

Consolidating into a single database eliminates data silos and guarantees ACID consistency across the entire application domain. With two disparate databases (Access and SQL Server), linking an employee record to the user who created it would require expensive, error-prone application-level joining across heterogeneous database connections. In contrast, a unified database allows high-performance declarative SQL operations using a `LEFT JOIN`. This single atomic query effortlessly preserves legacy records where `CreatedBy IS NULL` while displaying the creator's username for all active employees. Centralizing into SQL Server also provides robust connection pooling, automated disaster recovery, and unified security policies.

8. Real Build Errors Encountered & Solutions

1. **CS0017 - Multiple Entry Points Defined:**

Problem: Importing files from the Login project initially brought in a second `Program.cs`, resulting

in compiler error CS0017.

Solution: Deleted the secondary `Program.cs` and unified the startup routine in the host project's `Program.cs` to launch `frmLogin`.

2. Partial Class Namespace Mismatch:

Problem: Modifying `frmLogin.cs` namespace without updating `frmLogin.Designer.cs` caused compilation failure because partial classes must reside in identical namespaces.

Solution: Refactored namespaces across all form files simultaneously to `EmployeeDetails`.

3. Architecture Mismatch with Legacy Jet OLEDB:

Problem: Microsoft.Jet.OLEDB.4.0 is a 32-bit provider that fails on 64-bit PowerShell and .NET runtimes.

Solution: Used 32-bit interop to extract the Access user records, migrated them into SQL Server LocalDB, and completely removed OleDb from the application.

9. Bonus Features Implemented (+ 15 Marks)

- **SHA-256 Password Hashing & Legacy Auto-Upgrade:** Passwords for new registrations are securely hashed using SHA-256. Migrated plaintext accounts from Access can log in seamlessly; upon their first successful authentication, their password is automatically upgraded to a secure SHA-256 hash in the database.
- **Employee Search by Name / ID:** Parameterized search bar supporting `WHERE e.EmpName LIKE @term OR e.EmpId LIKE @term` with "Search" and "Show All" buttons.
- **Delete Confirmation Dialog:** Critical deletion operations require explicit user confirmation via `MessageBoxButtons.YesNo` to prevent accidental data loss.
- **Audit Activity Logging:** User logins and logouts are stamped with exact timestamps in table `dbo.LoginHistory`.

10. Conclusion & Verification Summary

The merged solution `24-57480-2_CompanyApp` satisfies all requirements set forth in the lab specification:

- ✓ Single project, single `.exe`, single startup entry point in `Program.cs`.
- ✓ Complete removal of `System.Data.OleDb` and `.mdb` dependencies.
- ✓ Unified SQL Server database `dbCompanyApp` with `Schema.sql`.
- ✓ Stamped ownership via `CreatedBy` foreign key and DataGridView `LEFT JOIN`.
- ✓ Implemented SHA-256 hashing, employee search, delete confirmation, and audit logging.
- ✓ Clean repository with `.gitignore`, `README.md`, and `Schema.sql`.

Lab 2 Final Report • Student ID: 24-57480-2 • CompanyApp Merged System